

Capítulo 1

Resultados

El propósito de este capítulo es reconstruir, paso a paso, cómo se construye un detector de alucinaciones para sistemas RAG con aprendizaje automático clásico, y presentar los resultados que alcanza. El recorrido sigue el orden natural de un problema de aprendizaje supervisado: primero se presenta la base de datos y se caracteriza el fenómeno que se quiere detectar; después se transforman los textos en variables mediante ingeniería de características y se selecciona un subconjunto con un método formal; a continuación se fijan las métricas y la metodología de evaluación; luego se entrena y compara cada modelo y se elige un ganador; y por último se sitúa el resultado frente al estado del arte. Dos aspectos reciben atención especial por su peso en este problema concreto: la *ingeniería de características*, donde reside casi toda la señal, y el *desbalance de clases*, que condiciona la decisión final. La exploración de las vías que no funcionaron se recoge, condensada, al final del capítulo, porque forma parte de la caracterización honesta del problema y no del hilo principal.

1.1. Introducción al problema

Un sistema de generación aumentada por recuperación (RAG, *Retrieval-Augmented Generation*) responde a una consulta en dos fases: primero *recupera* uno o varios documentos relevantes de una base documental, y después, condicionado a ellos, *genera* una respuesta en lenguaje natural [14]. La promesa de RAG es anclar la respuesta en fuentes concretas para reducir las invenciones del modelo. Sin embargo aparece un fallo específico: el sistema *alucina* cuando la respuesta afirma algo que el documento recuperado no respalda [12].

Este trabajo aborda solo la segunda fase. Se da la recuperación por hecha y se estudia el paso de *fidelidad* (*faithfulness*): dado el par (respuesta, fuente), decidir si la respuesta contiene alguna alucinación. No se construye ningún recuperador; la fuente es el documento que RAGTruth proporciona ya emparejado con la respuesta. Detectar estas alucinaciones importa porque un sistema que inventa datos con tono seguro es peligroso en aplicaciones reales. Hoy se detectan sobre todo con un modelo de lenguaje grande actuando de juez [22], lo que es caro, lento y opaco. El objetivo de este trabajo es construir un detector *clásico*, barato, determinista e interpretable, que use solo características léxicas y estadísticas y un clasificador clásico. La hipótesis falsable que guía el trabajo es que tal detector alcanza un AUC-ROC de al menos 0,80 sobre RAGTruth bajo una evaluación honesta.

1.2. Presentación de la base de datos

1.2.1. Composición

Se emplea RAGTruth [15], el primer corpus a gran escala de salidas de modelos de lenguaje anotadas por humanos a nivel de *tramo* (*span*) según contengan o no alucinaciones. Consta de 17 790 respuestas generadas por seis modelos distintos (GPT-4, GPT-3.5-turbo, Llama-2-7B/13B/70B y Mistral-7B) sobre 2 965 *prompts*, y cubre tres tareas de naturaleza muy distinta:

- **QA** (respuesta a preguntas): dada una pregunta y unos pasajes recuperados, el modelo redacta una respuesta. La fuente es texto en prosa.
- **Data2txt** (generación a partir de datos): dado un registro estructurado en JSON de un negocio del Yelp Open Dataset, el modelo redacta una descripción. La fuente son campos estructurados.
- **Summary** (resumen): dado un documento (noticias de CNN/DailyMail), el modelo produce un resumen. La fuente es un documento largo.

Se trabaja con la partición oficial del corpus: 15 090 respuestas de entrenamiento y 2 700 de test. La partición oficial se respeta para poder comparar con el estado del arte, que se evalúa sobre ese mismo test.

1.2.2. Variables

Cada registro del corpus aporta las variables de origen del Cuadro 1.1. El hecho determinante para todo lo que sigue es que la entrada del problema es *texto*, no un vector numérico, lo que obliga a la ingeniería de características de la sección 1.3.

Variable	Descripción
<code>response</code>	Respuesta generada por el modelo de lenguaje (el texto a auditar).
<code>source</code>	Documento fuente recuperado (el contexto que debe respaldar la respuesta).
<code>task_type</code>	Tarea: QA, Data2txt o Summary.
<code>model</code>	Modelo de lenguaje que generó la respuesta.
<code>hallucination_labels</code>	Anotación humana: lista de tramos (<i>spans</i>) alucinados, con su tipo.

Tabla 1.1: Variables de origen de RAGTruth. La variable objetivo se deriva de `hallucination_labels` (sección 1.2.3).

1.2.3. Variable objetivo

La anotación `hallucination_labels` marca, dentro de cada respuesta, los tramos concretos que alucinan y de qué tipo son. A partir de ella se define la variable objetivo binaria a nivel de respuesta: se etiqueta `label = 1` (alucina) si la respuesta contiene al menos un tramo anotado, y `label = 0` (limpia) en caso contrario. La anotación aparece en el corpus

en tres formatos (lista vacía, lista con tramos, o cadena de texto que representa la lista); los tres se normalizan al construir la etiqueta. La tarea es, por tanto, de **clasificación binaria a nivel de respuesta**. Se elige el nivel de respuesta, y no el de tramo, porque es la formulación que también usa el estado del arte con el que se comparará (sección 1.8), lo que hace las cifras directamente comparables.

1.2.4. Descriptivo y caracterización de las alucinaciones

Nivel global

Un ejemplo concreto fija las ideas antes que cualquier estadística. El Cuadro 1.2 muestra un par (fuente, respuesta) de cada clase. En el caso limpio, cada afirmación de la respuesta se puede rastrear hasta la fuente; en el caso alucinado, la respuesta introduce un dato que la fuente no contiene.

Fuente (datos estructurados)	«... 'categories': 'Gluten-Free, Gastropubs, Vegan, Restaurants, Indian', 'OutdoorSeating': True, ' WiFi ': 'no', 'RestaurantsTakeOut': True ...»
Resp. limpia (label=0)	«Apna Indian Kitchen is a local business located at 718 State St in Santa Barbara, CA. It offers a variety of options for customers, including gluten-free, gastropubs, vegan, and traditional Indian cuisine.»
Resp. alucinada (label=1)	«... The restaurant has a cozy atmosphere with outdoor seating and free WiFi Customers have given the restaurant a 4.5-star rating ...»

Tabla 1.2: Un par real de Data2txt sobre el mismo restaurante (Apna Indian Kitchen). La respuesta limpia se rastrea entera hasta la fuente; la alucinada afirma «**free WiFi**» cuando el dato estructurado dice 'WiFi': 'no' (span anotado como *Evident Conflict*), e introduce una valoración de 4,5 estrellas que la fuente no contiene.

A nivel global, la prevalencia de alucinación es del 44,5 % en entrenamiento y del 34,9 % en test: casi una de cada tres respuestas del corpus contiene alguna invención. La longitud de las respuestas y su distribución por tarea aparecen en la Figura 1.1; conviene retenerla porque la longitud entrará luego como característica.

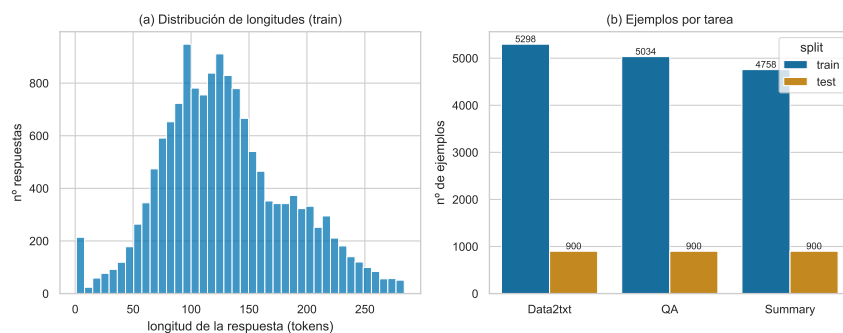


Figura 1.1: Descriptivo global de RAGTruth: distribución de longitudes de respuesta y número de ejemplos por tarea en entrenamiento y test.

Nivel por tareas

El desbalance global es moderado, pero engaña: al abrirlo por tarea (Cuadro 1.3 y Figura 1.2) aparece la estructura real. Data2txt es mayoritariamente positiva (dos de cada tres respuestas alucinan); QA y Summary son minoritariamente positivas, y su prevalencia *cae todavía más* en el test. Estamos ante un desbalance **heterogéneo por tarea** y con **desplazamiento** entre entrenamiento y test. La consecuencia práctica, que un único umbral de decisión no puede ser óptimo para prevalencias tan dispares, se aborda en la metodología (sección 1.5.4).

Tarea	prevalencia (train)	prevalencia (test)
Data2txt	0,694	0,643
QA	0,311	0,178
Summary	0,311	0,227
Global	0,445	0,349

Tabla 1.3: Prevalencia de alucinación por tarea. El desbalance difiere entre tareas y se desplaza train→test de forma desigual (el mayor salto es QA, 0,311 → 0,178).

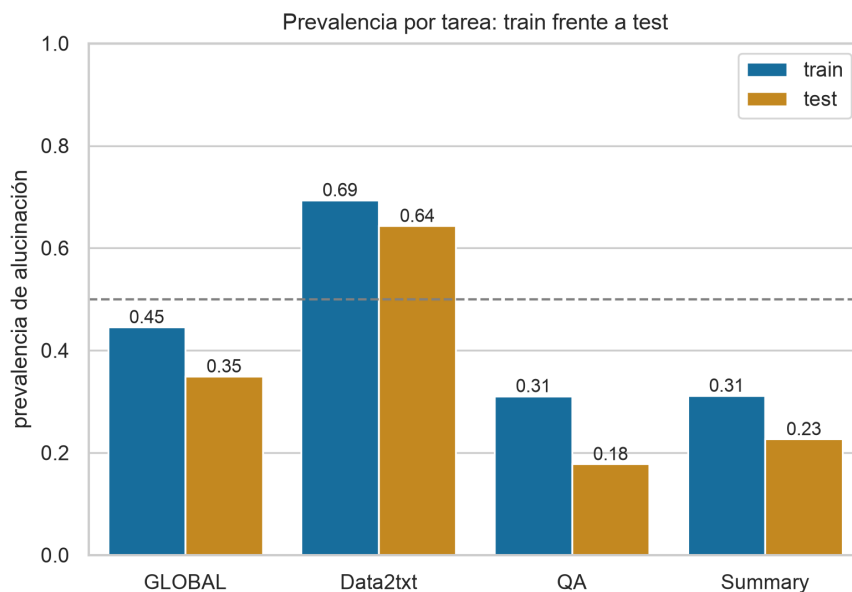


Figura 1.2: Prevalencia por tarea en entrenamiento y test.

Caracterización: un caso que el LLM-juez falla y el detector clásico acierta

Para motivar por qué un detector léxico tiene sentido conviene mostrar un caso donde *gana* frente al método dominante (el LLM-juez). Considérese el patrón más frecuente de Data2txt: la introducción de una cifra sin base. La Figura 1.3 lo esquematiza. La fuente no contiene el número 4,7; la respuesta lo afirma. Un juez basado en un modelo de lenguaje evalúa la respuesta por su *plausibilidad* global, y una valoración de 4,7 estrellas para una cafetería es perfectamente plausible, de modo que el juez tiende a racionalizarla y dejarla pasar; este sesgo de racionalización de los LLM-juez está documentado [17, 22], y se refleja

en que el juez GPT-4-turbo solo alcanza un F1 de 78,3% en Data2txt [15]. En cambio, una característica léxica trivial, la fracción de números de la respuesta presentes en la fuente (`num_overlap`), decide por *pertenencia a un conjunto*: como $4,7 \notin F$, la marca al instante, sin razonar sobre plausibilidad. En la subclase de alucinaciones *evidentes sin base* de tipo numérico, que domina en Data2txt, la aritmética de conjuntos es más fiable que el juicio semántico, y ahí el detector clásico es competitivo o superior a coste nulo.

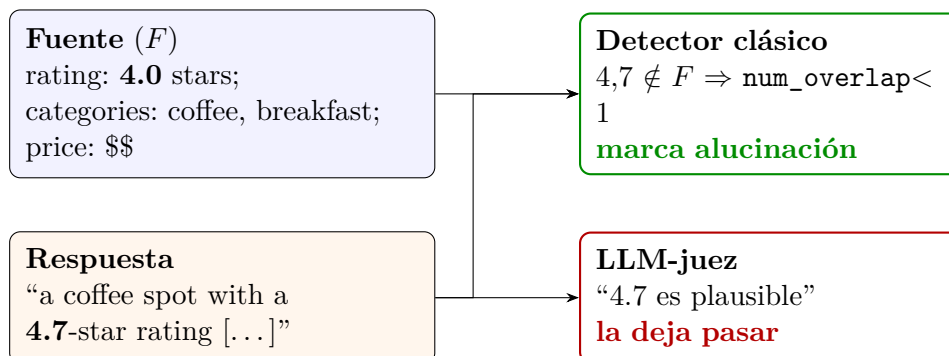


Figura 1.3: El mismo caso ante dos detectores. La respuesta inventa la cifra 4,7 (la fuente dice 4,0). El LLM-juez la evalúa por plausibilidad y la racionaliza; el detector clásico la marca por pertenencia a conjuntos ($4,7 \notin F$). Esquema del mecanismo, no de un ejemplo concreto anotado.

Distribución de las alucinaciones

RAGTruth distingue cuatro tipos de alucinación, cruzando dos ejes: si el error es *evidente* o *sutil*, y si consiste en información *sin base* (inventada) o en un *conflicto* directo con la fuente. Un ejemplo real de conflicto anotado en el corpus: ante una fuente que dice «*lowest 75% or highest 25%*», una respuesta que afirma «*highest 75%*» queda marcada como conflicto evidente [15]. El Cuadro 1.4 muestra la composición por tarea. El dato relevante para lo que sigue es que Summary es en su gran mayoría de tipo *evidente* ($\approx 86\%$) y aun así, como se verá, es la tarea más difícil de detectar: el tipo anotado no explica la dificultad. El descriptivo de los tramos (Figura 1.4) completa el retrato: dominan los números, las fechas y los nombres propios, lo que justifica de forma natural las características de solape numérico y de entidades, sin entrar todavía en su detalle (eso es la sección 1.3).

Tarea	Evidente sin base	Evidente conflicto	Sutil sin base	Sutil conflicto
Data2txt	0,383	0,447	0,163	0,007
QA	0,522	0,146	0,314	0,018
Summary	0,514	0,343	0,100	0,043

Tabla 1.4: Composición de los tipos de alucinación por tarea (fracción de tramos). Summary es mayoritariamente evidente y aun así la tarea difícil.

1.3. Ingeniería de características

Como la entrada es texto, el primer paso es transformar cada par (respuesta, fuente) en un vector numérico de características que capturen indicios de alucinación. Esta sección

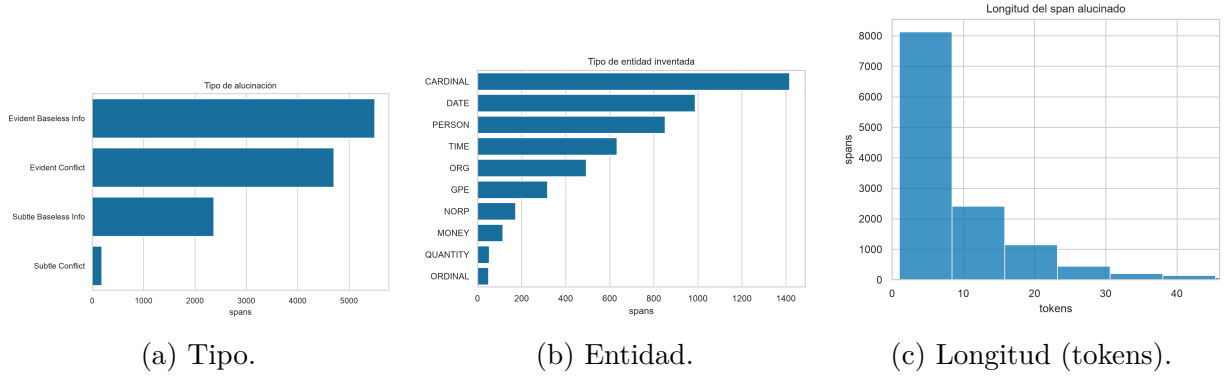


Figura 1.4: Descriptivo de los tramos alucinados. Predominan números y entidades nombradas.

es el núcleo del trabajo: define el preprocesado, el catálogo de características candidatas, comprueba si separan las clases y selecciona un subconjunto con un método formal.

1.3.1. Preprocesado: tokenización y notación

Toda característica se construye sobre una tokenización simple: el texto se pasa a minúsculas y se extraen las secuencias alfanuméricas como *tokens*. Para las características numéricas se usa una variante que conserva los decimales como una sola pieza (“26,2” es un token), necesaria para los n -gramas y la consistencia numérica. Sobre esta tokenización se definen los dos conjuntos básicos.

Definición 1. Dada una respuesta y su fuente, sea R el conjunto de palabras únicas de la respuesta y F el conjunto de palabras únicas de la fuente.

R y F son la base de casi todas las características. La idea recurrente es que una respuesta fiel reutiliza el vocabulario de la fuente, mientras que una alucinación introduce términos nuevos o recombina los existentes de otra forma.

1.3.2. Catálogo de las dieciocho características candidatas

Se construyen dieciocho características candidatas, agrupadas en cuatro familias (Cuadro 1.5). La **familia de solape léxico** mide cuánto vocabulario comparte la respuesta con la fuente; la **numérica y relacional** busca cifras y unidades fabricadas; la **de nivel frase** agrega el soporte frase a frase, con la intuición de que una alucinación suele ser *una* frase sin respaldo que el promedio global diluye, por lo que se agrega con el mínimo (la peor frase delata); y la **codificación one-hot de la tarea** permite al modelo especializar su decisión por tipo de tarea. Se definen a continuación las cinco básicas, que son las que aparecen en el análisis; el resto son variantes (n -gramas de orden 2 y 3, agregados por frase) que se leen del cuadro.

Containment (precisión léxica).

$$\text{containment} = \frac{|R \cap F|}{|R|}. \quad (1.1)$$

Fracción de palabras distintas de la respuesta que también están en la fuente. Alto significa que todo lo que afirma la respuesta tiene respaldo léxico; bajo, que introduce palabras nuevas. Es, casi literalmente, la definición de *groundedness* convertida en número.

Jaccard.

$$\text{jaccard} = \frac{|R \cap F|}{|R \cup F|}. \quad (1.2)$$

Solape simétrico: mismo numerador que **containment** pero dividido por la unión, de modo que penaliza también el vocabulario de la fuente que la respuesta no usa.

Coseno TF-IDF. El coseno entre los vectores TF-IDF [20] de respuesta y fuente,

$$\cos(u, v) = \frac{\sum_t u_t v_t}{\sqrt{\sum_t u_t^2} \sqrt{\sum_t v_t^2}}, \quad u_t = \text{tf}(t) \cdot \log \frac{N}{\text{df}(t)}, \quad (1.3)$$

que pondera cada término por su rareza: las palabras comunes pesan poco y las específicas, mucho. Mide similitud distribucional, no solo solape de conjuntos.

Solape numérico.

$$\text{num_overlap} = \frac{|N_R \cap N_F|}{|N_R|} \quad (= 1 \text{ si } N_R = \emptyset), \quad (1.4)$$

con N_R, N_F los conjuntos de números de respuesta y fuente. Caza fechas y cifras inventadas; si la respuesta no tiene números, no hay nada que pueda estar sin respaldo y vale 1.

Longitud de la respuesta. $\text{answer_len} = |\text{tokens}(R)|$, el número de palabras de la respuesta (con repeticiones). No indica alucinación por sí sola, pero contextualiza a las demás (un **containment** de 0,8 no significa lo mismo en 3 palabras que en 40).

1.3.3. ¿Separan las clases? Separabilidad y correlación

Antes de seleccionar conviene ver dos cosas: si las características discriminan, y cuánta información *independiente* aportan. La Figura 1.5 muestra la distribución de cada característica continua separada por la variable objetivo: la señal es visible, en **containment**, **jaccard** o **num_context** la clase alucinada se desplaza hacia menos solape.

Ahora bien, discriminar no basta: dos características muy correlacionadas dicen lo mismo. La Figura 1.6 muestra la correlación de Spearman entre las dieciocho. El patrón es contundente: **containment** correlaciona por encima de 0,8 (en valor absoluto) con **tfidf_cos**, **sent_cont_mean**, **sent_sim_mean**, **novel_bigram** y **novel_trigram**, y estas dos últimas son casi el negativo exacto de **containment**. En otras palabras, **las dieciocho características no codifican dieciocho señales independientes, sino unos pocos ejes ortogonales**:

1. **Soporte léxico:** **containment**, **jaccard**, **tfidf_cos**, novedad de n -gramas, agregados por frase. Todas miden lo mismo (cuánto se apoya la respuesta en la fuente). Representante natural: **containment**.
2. **Consistencia numérica:** **num_overlap** y **num_context**, poco correlacionadas con el eje anterior. Miden si las cifras y sus unidades están respaldadas.
3. **Longitud:** **answer_len**, ortogonal al resto.
4. **Polaridad:** **neg_diff**, señal débil pero independiente.

Característica	Definición	Señal que captura
<code>containment</code>	$ R \cap F / R $	precisión léxica (señal dominante)
<code>jaccard</code>	$ R \cap F / R \cup F $	solape simétrico
<code>tfidf_cos</code>	coseno TF-IDF respuesta-fuente	similitud distribucional
<code>num_overlap</code>	fracción de números de R en F	cifras/fechas inventadas
<code>novel_bigram</code>	fracción de bigramas de R no en F	recombinación de pares
<code>novel_trigram</code>	ídem con trigramas	recombinación estricta
<code>num_context</code>	fracción de números con su unidad en F	cambio de unidad/atributo
<code>neg_diff</code>	diferencia de densidad de negación	conflicto de polaridad
<code>answer_len</code>	longitud de la respuesta (tokens)	longitud
<code>sent_cont_*</code>	containment por frase (mín/media/desv/% bajo)	frase peor sostenida
<code>sent_sim_*</code>	similitud por frase (mín/media)	frase menos parecida
<code>task_*</code>	one-hot de la tarea (QA/Summary/Data2txt)	especialización por tarea

Tabla 1.5: Las dieciocho características candidatas. R y F son los conjuntos de palabras únicas de respuesta y fuente. Las familias son: solape léxico, numérica/relacional, nivel frase y one-hot de tarea.

Esta observación es la que motiva y anticipa la selección: si hay tres o cuatro ejes, sobran variables, y conservarlas todas solo añade ruido colineal que perjudica sobre todo a las tareas de poca señal. La regla que se aplicará es podar por *redundancia* (el heatmap), no por correlación con la etiqueta.

1.3.4. Selección de variables con RFE

Métodos de selección: filtro y envolvente

Los métodos de selección de características se dividen en dos grandes familias [10]. Los **métodos de filtro** puntúan cada característica por una medida estadística de su relación con la etiqueta (correlación, información mutua, χ^2) *independientemente del modelo*; son rápidos, pero ignoran las interacciones y la redundancia entre variables (dos características redundantes puntúan alto las dos). Los **métodos envolventes** (*wrapper*) usan el propio modelo como juez: prueban subconjuntos, los entrenan y se quedan con el que mejor valida; son más caros pero capturan interacciones y redundancia. Dado que aquí el problema es justamente la redundancia (sección 1.3.3), se usa un método envolvente.

RFE y el método del codo

Recursive Feature Elimination (RFE) [11] es el método envolvente elegido: se entrena el modelo con todas las características, se elimina la menos importante según la importancia

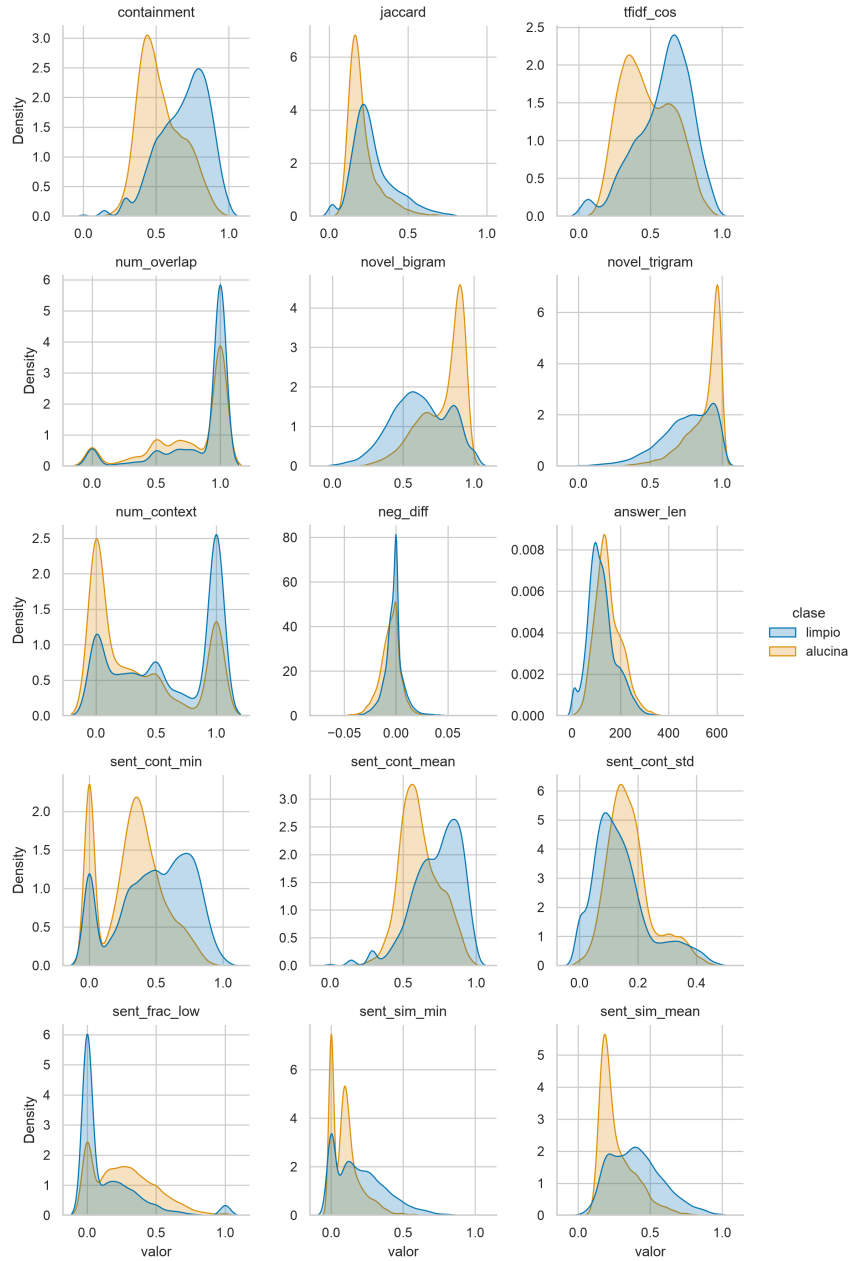


Figura 1.5: Distribución de cada característica continua separada por clase. `containment` y las medidas de solape muestran el mayor desplazamiento.

interna del modelo, se reentrena y se repite hasta agotar las variables. El orden inverso de eliminación produce un *ranking* de más a menos importante. A continuación se traza el rendimiento en validación cruzada (GroupKFold por `source`, sección 1.5.2) frente al número de variables añadidas en ese orden, y se fija el tamaño por el **método del codo** sobre la curva de F1: el punto a partir del cual añadir variables deja de aportar.

La Figura 1.7 muestra la curva. El AUC sube de golpe hasta unas seis variables y luego se aplana; el F1 alcanza su mejor valor en $k = 11$ y a partir de ahí no mejora. Se fija el conjunto en **once variables**. El coseno TF-IDF no entra en la selección (su aporte es de cola, por debajo de 0,004 de importancia), y se acepta el veredicto del método sin forzarlo, coherente con que es redundante con `containment` (sección 1.3.3).

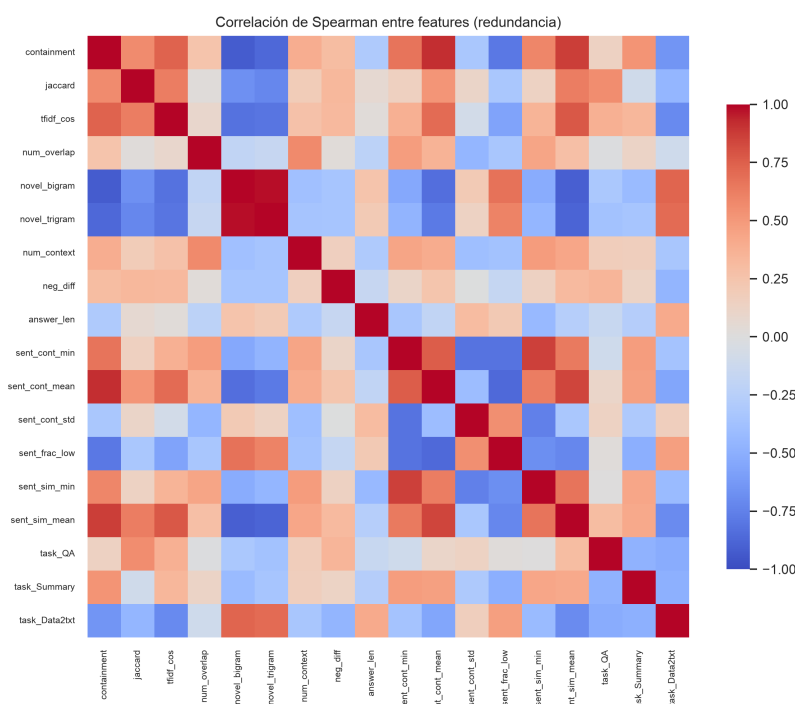


Figura 1.6: Correlación de Spearman entre las dieciocho características. El bloque de solape léxico está muy correlacionado ($|\rho| > 0,8$): pocas señales independientes.

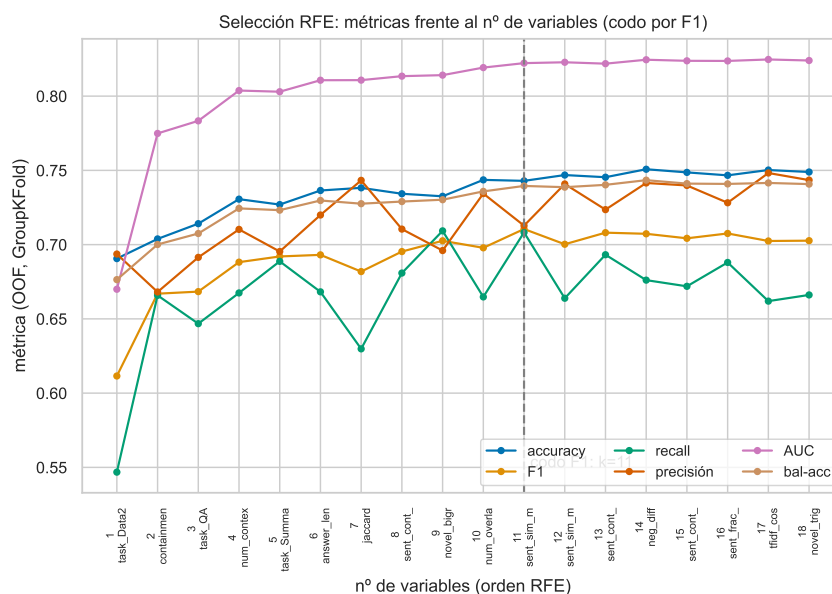


Figura 1.7: Selección RFE: las seis métricas (OOF, GroupKFold) frente al número de variables, en orden RFE. El codo sobre F1 fija $k = 11$; el rango 6–18 está prácticamente empatado en AUC.

El conjunto elegido y por qué

Las once variables definitivas, en orden de importancia RFE, son:

`containment`, `task_QA`, `task_Summary`, `task_Data2txt`, `num_context`, `answer_len`, `jaccard`,
`sent_cont_min`, `novel_bigram`, `num_overlap`, `sent_sim_min`.

El conjunto es *interpretable* y respeta los ejes de la sección 1.3.3: `containment` domina (el eje de soporte léxico); le acompañan la consistencia numérica (`num_context`, `num_overlap`), la recombinación (`novel_bigram`), el soporte por frase (`sent_cont_min`, `sent_sim_min`), la longitud, y la codificación de tarea. Cada variable elegida representa un eje distinto o aporta un matiz que RFE juzgó no redundante. La Figura 1.8 muestra la separabilidad por clase y la correlación de Spearman del conjunto final: la señal por clase se mantiene y la redundancia ha bajado respecto a las dieciocho. Sobre este conjunto se hace todo el estudio posterior.

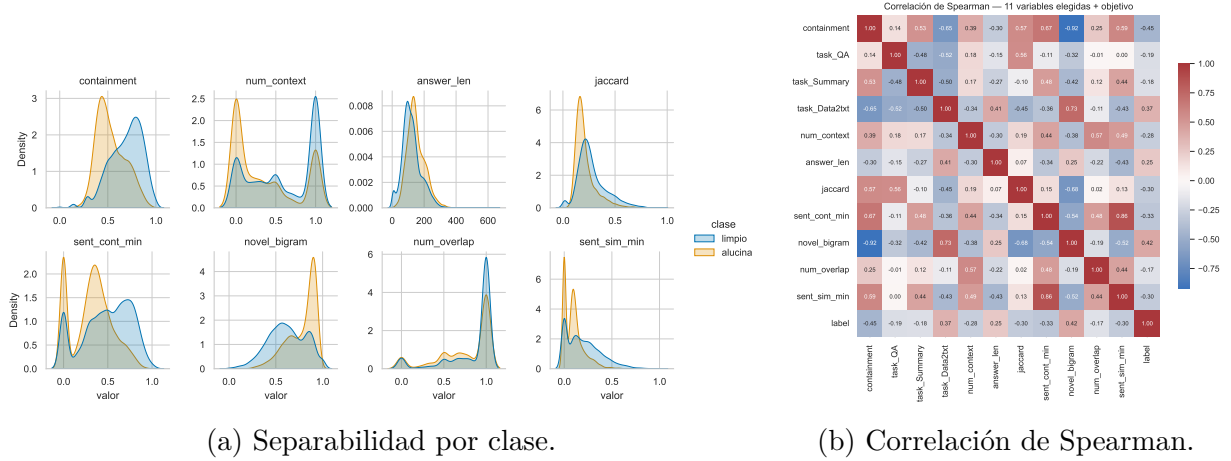


Figura 1.8: Descriptivo del conjunto de once variables elegido por RFE.

1.4. Métricas de evaluación

Antes de la metodología se fijan las métricas, porque toda decisión posterior (umbral, modelo ganador) se toma con ellas. Todas se construyen sobre los cuatro conteos de la matriz de confusión (Figura 1.9), tomando como clase positiva la alucinación: verdaderos positivos (VP), verdaderos negativos (VN), falsos positivos (FP, error de tipo I) y falsos negativos (FN, error de tipo II). En detección de alucinaciones el FN es el error caro: una alucinación que se cuela sin detectar.

De estos conteos se definen la precisión y la exhaustividad (recall):

$$\text{precisión} = \frac{VP}{VP + FP}, \quad \text{recall} = \frac{VP}{VP + FN}. \quad (1.5)$$

La precisión mide cómo de acertado es el modelo cuando predice alucinación; el recall, cuántas de las alucinaciones reales caza. Ambas se mueven en sentidos opuestos según el umbral: exigir más para predecir positivo sube la precisión y baja el recall. La curva precisión-recall resume ese equilibrio, y a diferencia de la ROC *sí* refleja el desbalance, porque la precisión depende de la prevalencia (su línea base está en $y = \text{prevalencia}$).

		Predicción	
		limpia	alucina
Realidad	alucina	VP (alucina bien detectada)	FP (falsa alarma) Error tipo I
	limpia	FN (alucinación no vista) Error tipo II	VN (limpia bien detectada)

Figura 1.9: Matriz de confusión. La clase positiva es la alucinación; el falso negativo (tipo II) es una alucinación no detectada.

La curva ROC contrapone la tasa de verdaderos positivos (el recall) frente a la tasa de falsos positivos al variar el umbral, y su área, el AUC-ROC [8], resume el rendimiento en *todos* los umbrales. El AUC es la probabilidad de que una alucinación tomada al azar puntúe más alto que una respuesta limpia tomada al azar: mide la separación pura, y es *invariante al umbral y a la prevalencia*. Por eso es la métrica interna del trabajo (cumple la hipótesis $\geq 0,80$ y es estándar), pero *no* basta para juzgar si el detector caza las alucinaciones a un umbral fijo: eso lo mide el recall. El valor F1 combina precisión y recall en su media armónica, y su generalización F_β pondera el recall β^2 veces más que la precisión:

$$F_1 = 2 \frac{\text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}, \quad F_\beta = (1 + \beta^2) \frac{\text{precisión} \cdot \text{recall}}{\beta^2 \text{precisión} + \text{recall}}. \quad (1.6)$$

Se reporta el AUC como métrica interna, y F1 y accuracy para comparar con el estado del arte, que se mide en esas dos. No se adopta F_β como métrica principal: elegir β equivale a fijar un ratio de coste FN/FP que solo tiene sentido ante un despliegue concreto, del que este estudio carece; en su lugar la prioridad del recall se expresa en la capa de decisión (el punto de operación, sección 1.7.3).

1.5. Metodología de entrenamiento y validación

Esta sección describe el flujo desde la base de datos en crudo hasta la evaluación final (Figura 1.10) y los tres problemas que hay que resolver para que la evaluación sea honesta.

1.5.1. División entrenamiento / test

Se respeta la partición oficial de RAGTruth: 15 090 respuestas de entrenamiento y 2 700 de test. Todas las decisiones (características, hiperparámetros, umbral) se toman usando *solo* el entrenamiento, mediante validación cruzada interna; el test oficial permanece intacto durante todo el desarrollo y se evalúa una única vez, para la cifra final. Esta

Flujo de trabajo: del dato en crudo a la evaluación honesta

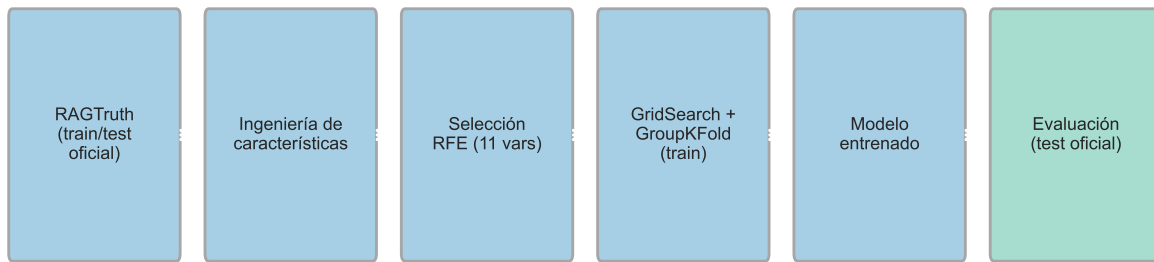


Figura 1.10: Flujo de trabajo: del dato en crudo a la evaluación en el test oficial.

disciplina evita el sobreajuste al test por iteración, un riesgo real cuando se prueban muchas configuraciones.

1.5.2. Problema 1: dependencia de la partición (GroupKFold)

Las métricas dependen de cómo se parta el dato. Para no depender de una única partición se usa validación cruzada. Ahora bien, en RAGTruth varias respuestas comparten el mismo documento fuente, y si un documento aparece a la vez en entrenamiento y validación el modelo memoriza en lugar de generalizar, inflando el resultado. Por eso se usa **GroupKFold agrupando por source**: cada documento cae entero en un solo fold (Figura 1.11). Un KFold aleatorio filtraría información entre folds.

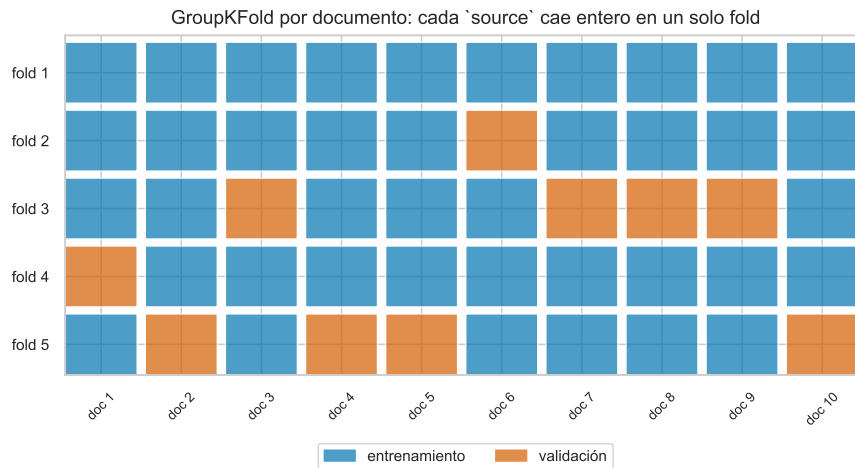


Figura 1.11: GroupKFold por documento: cada **source** cae entero en un fold, sin fuga entre entrenamiento y validación.

1.5.3. Problema 2: dependencia de los hiperparámetros (grid search)

Los modelos con hiperparámetros pueden sobreajustarlos. Para elegirlos se usa *grid search*: se prueban combinaciones de hiperparámetros y se conserva la de mejor F1 en validación cruzada, sin tocar el test. El coste es multiplicativo: optimizar tres hiperparámetros

con tres valores cada uno, con validación cruzada de cinco iteraciones, exige $3 \cdot 3 \cdot 3 \cdot 5 = 135$ ajustes del modelo. Este coste, asumible en CPU para modelos clásicos (minutos), es una de las ventajas del enfoque frente a las redes profundas, donde cada ajuste cuesta horas de GPU.

1.5.4. Problema 3: el desbalance y el punto de decisión

El tercer problema es propio de este trabajo. Como la prevalencia difiere por tarea (sección 1.2.3), un único umbral es subóptimo. La Figura 1.12 (izquierda) muestra cómo varían precisión, recall y F1 al mover el umbral. Se comparan tres criterios, todos con el umbral fijado en entrenamiento y aplicado al test: el índice de Youden global, el máximo de F1 por tarea y el máximo de F_2 por tarea. El umbral *por tarea* (Figura 1.12, derecha) reconoce que cada tarea opera a una prevalencia distinta y recupera falsos negativos donde el umbral global los dejaba pasar, a costa de precisión. Como se verá en la sección 1.7.3, recorta los falsos negativos de 285 a 125.

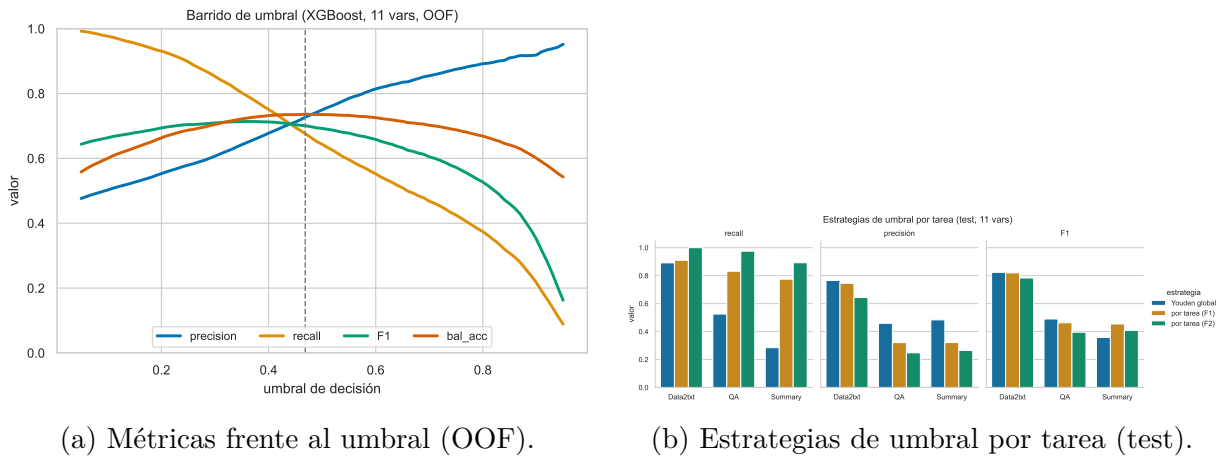


Figura 1.12: El umbral desliza el punto de operación entre precisión y recall; un umbral por tarea se adapta a cada prevalencia.

1.6. Marco de trabajo

El detector se implementa como una librería instalable en Python, `ragcheck` (`src-layout`, `pip install -e .`), que separa el motor reutilizable y testeado del conjunto de scripts atómicos que producen cada cifra y figura de este capítulo. La base de datos se descarga de forma programática del repositorio de RAGTruth alojado en Hugging Face [15], y se cachea localmente. La librería expone los módulos de carga y etiquetado (`data.py`), construcción de características (`features.py`), entrenamiento con validación cruzada (`training.py`), métricas (`evaluate.py`) e inferencia (`inference.py`), más una interfaz de línea de comandos (`ragcheck train|evaluate |score`). Los scripts de reentrenamiento y evaluación (uno por modelo, más la comparativa) son atómicos: cada cifra tiene una fuente única y trazable. Todo el trabajo se aloja en dos repositorios de git: el del *código* (público) y el de la *memoria* (privado). La reproducibilidad es total: semilla fija (42) en `numpy`, `scikit-learn` y `xgboost`, de modo que el mismo código y los mismos datos producen siempre el mismo resultado. Una batería de tests (incluido un test que verifica la ausencia de fuga en GroupKFold) protege el motor.

1.7. Resultados

1.7.1. Resultados por modelo

Se presenta el rendimiento de cada clasificador clásico sobre el conjunto de once variables. Para cada uno se indican su intuición, sus ventajas y desventajas en este problema, sus hiperparámetros óptimos (por grid search con GroupKFold) y sus métricas en el test oficial con el umbral de Youden. Las curvas ROC y PR conjuntas aparecen en la Figura 1.18.

Regresión logística. Modela la probabilidad de alucinación con una función sigmoide de una combinación lineal de las características [6]. Ventaja: máxima interpretabilidad (cada coeficiente es el peso de una característica) y coste ínfimo. Desventaja: solo captura fronteras lineales. La búsqueda en rejilla (Cuadro 1.6) elige regularización $C = 0,3$ y `class_weight = balanced`; alcanza AUC 0,806 y F1 0,658, con el recall más alto (0,749) por el reponderado de clases, a costa de precisión. Su curva ROC está en la Figura 1.13.

C	class_weight	F1 (CV)
0,3	balanced	0,673
0,1 / 1 / 3 / 10	balanced	0,672
0,1	—	0,663
0,3	—	0,661
1 / 3 / 10	—	0,657–0,658

Tabla 1.6: Búsqueda en rejilla de la regresión logística (F1 en validación cruzada, GroupKFold). El reponderado de clases (*balanced*) es lo que decide.

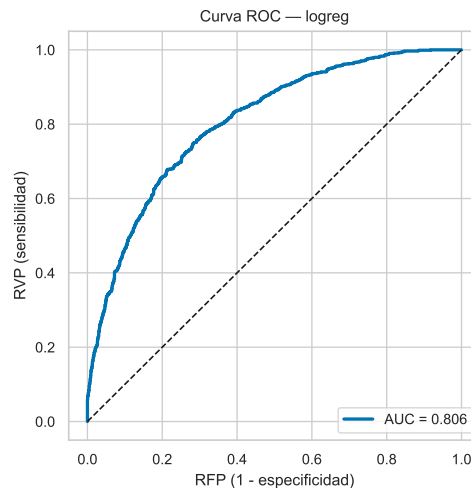


Figura 1.13: Curva ROC de la regresión logística (test, AUC 0,806).

K vecinos más cercanos (KNN). Clasifica según las clases de los k vecinos más próximos sobre las características estandarizadas [5]. Ventaja: no asume forma de la frontera. Desventaja: sensible a la escala y al ruido, y su inferencia guarda todo el entrenamiento. La rejilla (Cuadro 1.7) elige $k = 25$ vecinos ponderados por distancia; da AUC 0,826 y F1 0,681 (Figura 1.14).

k	pesos	F1 (CV)
25	distance	0,685
35	distance / uniform	0,684
25 / 15	uniform	0,684
15	distance	0,683
11	uniform / distance	0,681
5	uniform / distance	0,672

Tabla 1.7: Búsqueda en rejilla de KNN. La curva se satura a partir de $k \approx 15$; se toma $k = 25$ por dar el máximo.

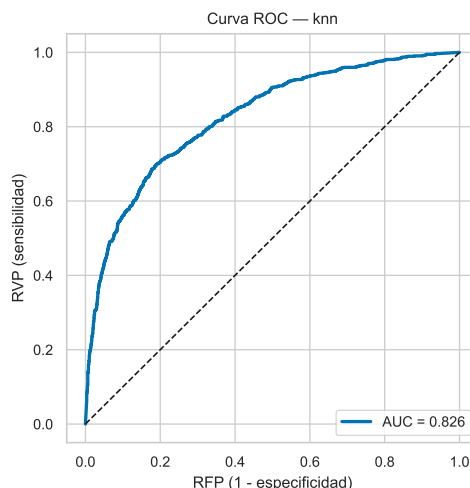


Figura 1.14: Curva ROC de KNN (test, AUC 0,826).

Máquina de vectores de soporte (SVM). Busca una frontera de máximo margen en un espacio transformado por un núcleo RBF [4]. Ventaja: fronteras no lineales con buen control del sobreajuste. Desventaja: no da probabilidades de forma nativa y escala mal con el número de ejemplos. La rejilla (Cuadro 1.8) elige $C = 10$; obtiene AUC 0,813 y el mejor F1 (0,690) y accuracy (0,784) de los cinco (Figura 1.15).

C	gamma	F1 (CV)
10	scale / auto	0,688
100	scale / auto	0,682
1	scale / auto	0,680

Tabla 1.8: Búsqueda en rejilla de la SVM (sobre submuestra por coste). **gamma** apenas influye; $C = 10$ es el mejor compromiso.

Bosque aleatorio. Promedia muchos árboles de decisión entrenados sobre submuestras [1]. Ventaja: robusto, capta interacciones, poco sensible a la escala. Desventaja: menos interpretable que un solo árbol. La rejilla (Cuadro 1.9) elige profundidad máxima 20; alcanza AUC 0,830 y F1 0,676 (Figura 1.16).

Gradient boosting (XGBoost). Construye árboles secuencialmente, cada uno corrigiendo el error del conjunto anterior [9, 3]. Ventaja: suele ser el mejor en datos tabulares y

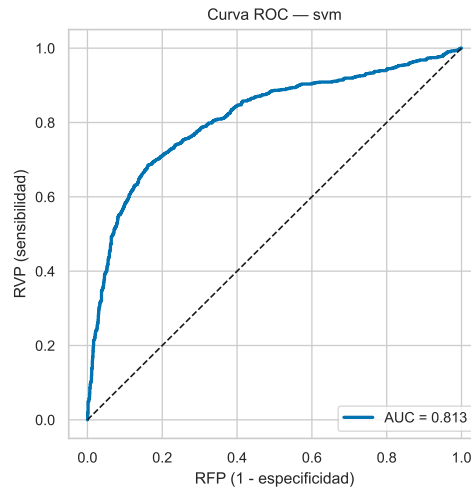


Figura 1.15: Curva ROC de la SVM (test, AUC 0,813).

prof. máxima	F1 (CV)
20	0,693
sin límite	0,691
12	0,689
8	0,684

Tabla 1.9: Búsqueda en rejilla del bosque aleatorio. Árboles profundos (20) rinden algo mejor que sin límite.

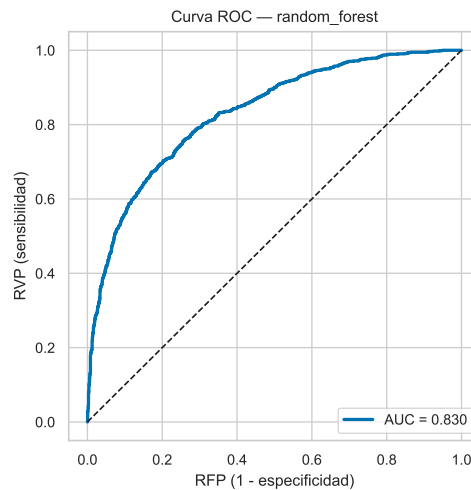


Figura 1.16: Curva ROC del bosque aleatorio (test, AUC 0,830).

da probabilidades bien calibradas. Desventaja: más hiperparámetros que ajustar. La rejilla (Cuadro 1.10) elige profundidad 6 y tasa de aprendizaje 0,05; es el mejor en AUC (0,837) (Figura 1.17).

1.7.2. Modelo ganador

Para elegir el mejor modelo se usa como criterio el promedio entre F1 y AUC, que equilibra la separación pura (AUC) con el rendimiento al umbral (F1). El Cuadro 1.11

tasa aprend.	prof. máx	F1 (CV)
0,05	6	0,692
0,1	4 / 6	0,692
0,05	4	0,691
0,1	3	0,690
0,05	3	0,689

Tabla 1.10: Búsqueda en rejilla de XGBoost. Todas las combinaciones rinden parecido; se toma prof. 6 y tasa 0,05.

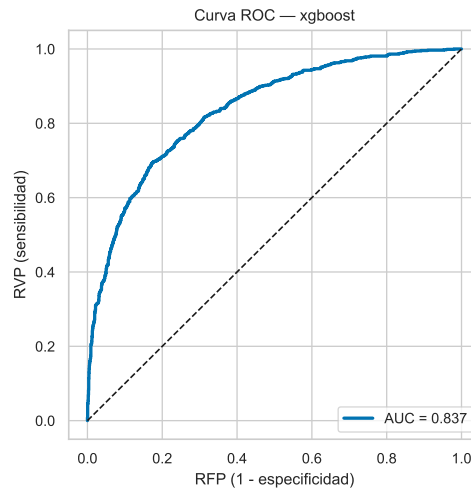


Figura 1.17: Curva ROC de XGBoost (test, AUC 0,837).

recoge las métricas de los cinco. Las diferencias son pequeñas y *los cinco superan el AUC 0,80 de la hipótesis*. El mejor por promedio F1–AUC es **XGBoost** (0,763), que combina el mejor AUC con buena calibración; será el modelo de referencia. Estas once variables igualan al conjunto completo de dieciocho (XGBoost con dieciocho da AUC 0,835), lo que confirma que la selección no pierde señal.

Modelo	AUC	F1	precisión	recall	accuracy	bal-acc	prom. F1–AUC
XGBoost	0,837	0,688	0,685	0,691	0,781	0,760	0,763
KNN	0,826	0,681	0,665	0,698	0,771	0,754	0,754
Bosque aleatorio	0,830	0,676	0,679	0,672	0,774	0,751	0,753
SVM	0,813	0,690	0,693	0,686	0,784	0,762	0,751
Reg. logística	0,806	0,658	0,587	0,749	0,729	0,733	0,732

Tabla 1.11: Comparación de los cinco modelos en el test oficial (once variables, umbral de Youden). Ganador por promedio F1–AUC: XGBoost.

1.7.3. El ganador por tarea: el techo de Summary y el compromiso recall–precisión

El desglose por tarea del ganador (Cuadro 1.12) revela dónde está la dificultad, algo que el número global esconde.

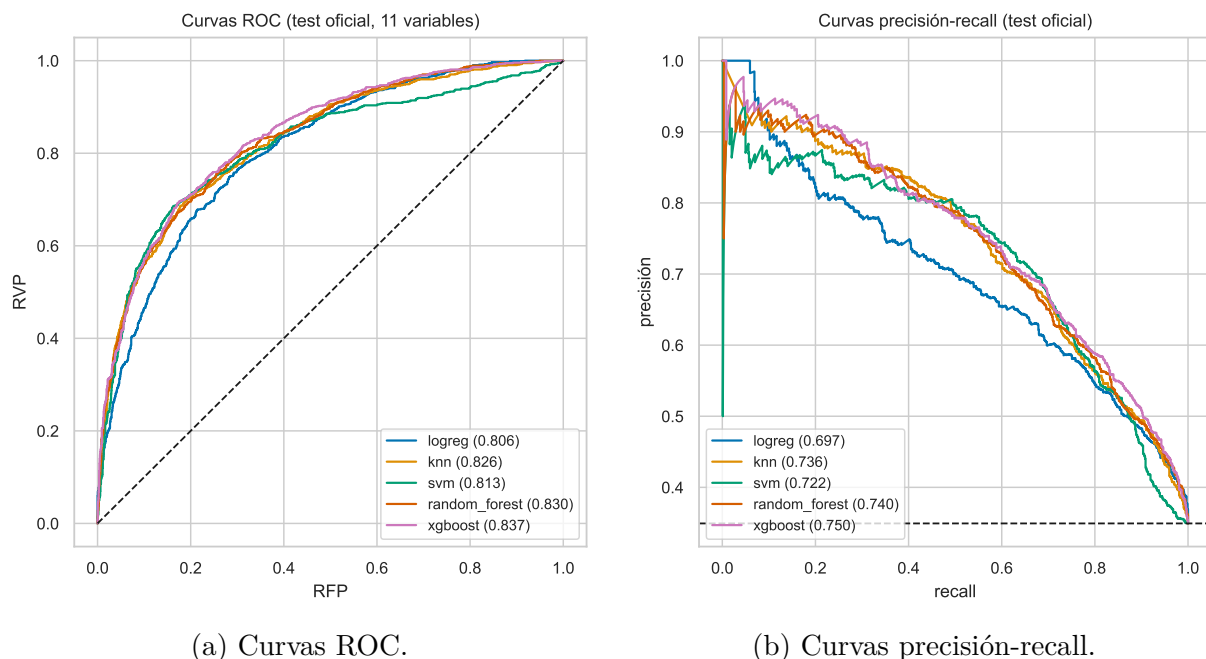


Figura 1.18: Discriminación de los cinco modelos sobre el test oficial (once variables).

Tarea	prev.	AUC	F1	recall	accuracy	FN
Data2txt	0,643	0,786	0,824	0,891	0,754	63
QA	0,178	0,817	0,490	0,525	0,806	76
Summary	0,227	0,702	0,358	0,284	0,769	146
Global	0,349	0,837	0,685	0,698	0,776	285

Tabla 1.12: XGBoost por tarea (test oficial, umbral de Youden global). QA discrimina bien (AUC 0,82) pero calla positivos por su baja prevalencia; Summary se queda en AUC 0,70.

El techo de Summary. Data2txt y QA se separan bien (AUC 0,79 y 0,82); Summary se queda en 0,70. No es un problema de umbral (el AUC es independiente del umbral) sino de *señal*. La causa está caracterizada: un resumen reutiliza el vocabulario del documento, así que el tramo alucinado comparte palabras con la fuente aunque el error sea evidente. El solape del tramo alucinado es de 0,62 de mediana en Summary frente a 0,43 en Data2txt, y *dentro* de Summary los falsos negativos tienen más solape (0,641) que los aciertos (0,555): el modelo falla justo en los tramos camuflados (Figura 1.24). Ninguna característica de superficie distingue el resumen fiel del alucinado cuando ambos recombinan el mismo vocabulario; es la frontera del enfoque léxico, no un defecto de implementación.

La paradoja de QA y el compromiso recall–precisión. QA tiene el mejor AUC (0,82) y a la vez un F1 modesto (0,49). La causa es la prevalencia: con solo un 18% de positivos, un umbral global calla los positivos y el recall se hunde (0,525). Es decir, el detector *ordena* bien pero, al umbral global, no *caza* lo suficiente. Aquí es donde el compromiso recall–precisión se vuelve explícito: subir el recall exige bajar la precisión, y cuánto se paga depende de la tarea. La Figura 1.19 muestra la curva PR por tarea con el punto de operación marcado. Su lectura cuantitativa es directa: para exigir un recall del 80%, en Data2txt la precisión apenas baja a **79,6%**, mientras que en Summary se desploma a **31,3%**. Ese es, medido y no supuesto, el precio de cazar las alucinaciones de

Summary con características léxicas.

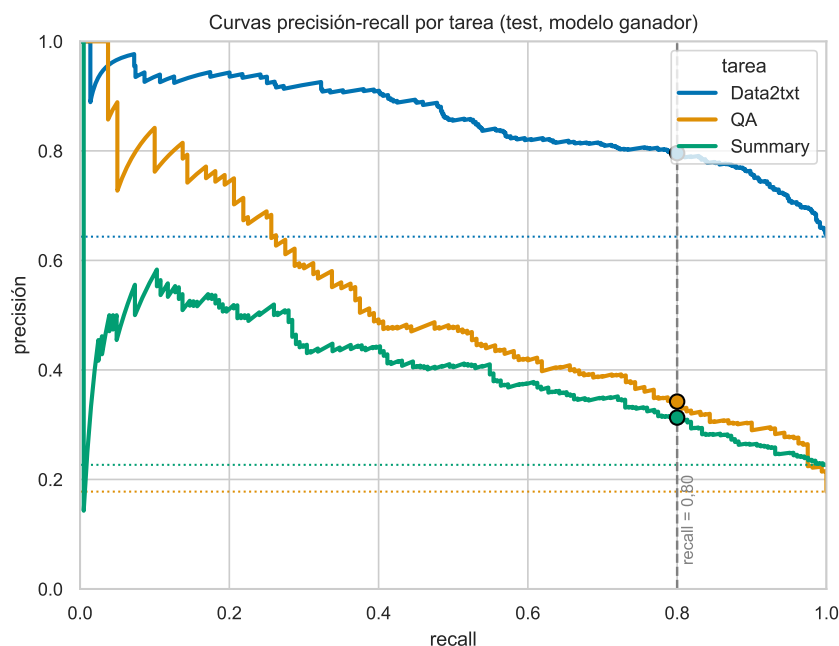


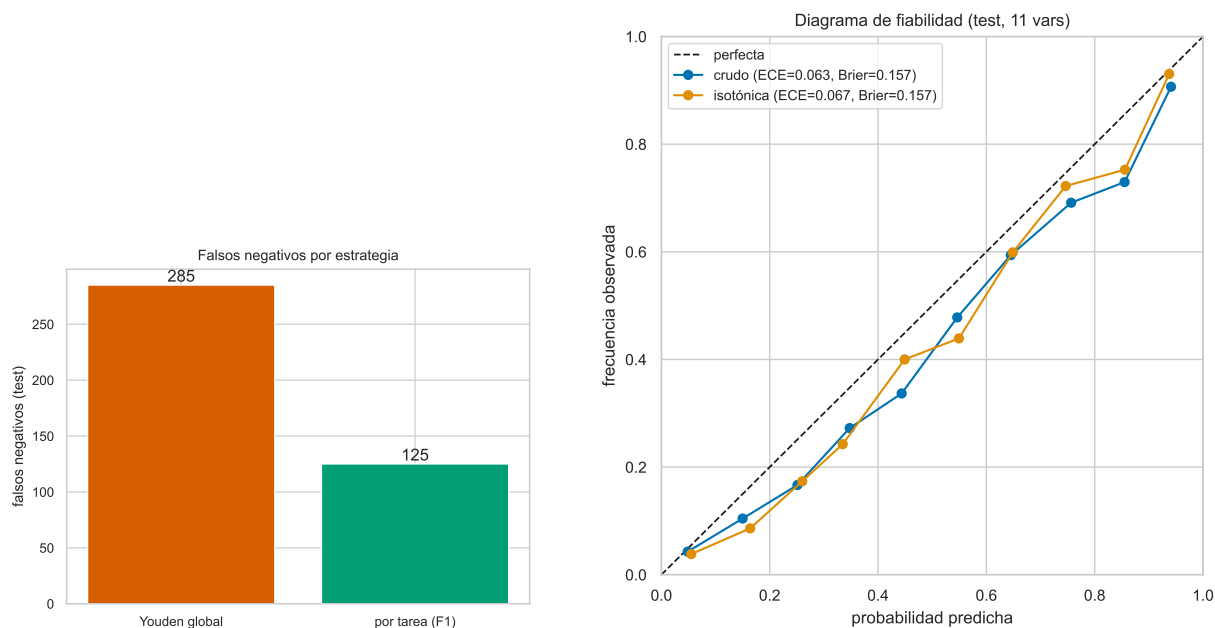
Figura 1.19: Curvas precisión-recall por tarea con la línea base en $y = \text{prevalencia}$. Fijar un recall del 80% cuesta poca precisión en Data2txt y mucha en Summary: ese es el compromiso rigor-precisión del detector.

El umbral por tarea. Reconocer que cada tarea opera a una prevalencia distinta permite fijar el umbral por tarea (sección 1.5.4). Al hacerlo, los falsos negativos totales bajan de 285 a 125 (Figura 1.20a), el recall de QA sube a 0,831 y el F1 de Summary de 0,358 a 0,454. La calibración del ganador es buena y mejora con regresión isotónica [16] (Figura 1.20b), lo que da probabilidades fiables sobre las que fijar el umbral. Las matrices de confusión por tarea (Figura 1.21) muestran visualmente el recorte de falsos negativos: la celda FN, las alucinaciones que se escapan, encoge en QA y Summary.

1.8. Comparación con el estado del arte

1.8.1. Los métodos de referencia

El estado del arte sobre RAGTruth se agrupa en tres familias, muy distintas del detector de este trabajo en tamaño y coste (Cuadro 1.13). Los **LLM-juez** usan un modelo de lenguaje grande con un *prompt* para emitir el veredicto (GPT-4-turbo, GPT-4o): no se entrenan, pero cuestan del orden de un céntimo y varios segundos por evaluación, y son opacos. Los **detectores fine-tuned** ajustan un LLM completo sobre RAGTruth (Llama-2-13B, Lynx-8B/70B, RAG-HAT): máximo rendimiento a cambio de miles de millones de parámetros y GPU. Los **encoders ligeros** entrenan un codificador pequeño (Luna; LettuceDetect, 150–396 M sobre ModernBERT): un punto intermedio, todavía neuronal y con GPU. Todos evalúan sobre el split oficial de RAGTruth y a nivel de respuesta; la mayoría trata el desbalance de forma implícita (el ajuste fino aprende la prevalencia) y no publica su desglose por tarea salvo los recopilados por Kovács et al.



(a) Falsos negativos por estrategia.

(b) Diagrama de fiabilidad.

Figura 1.20: El umbral por tarea recorta el falso negativo; la calibración es buena y mejora con isotónica.

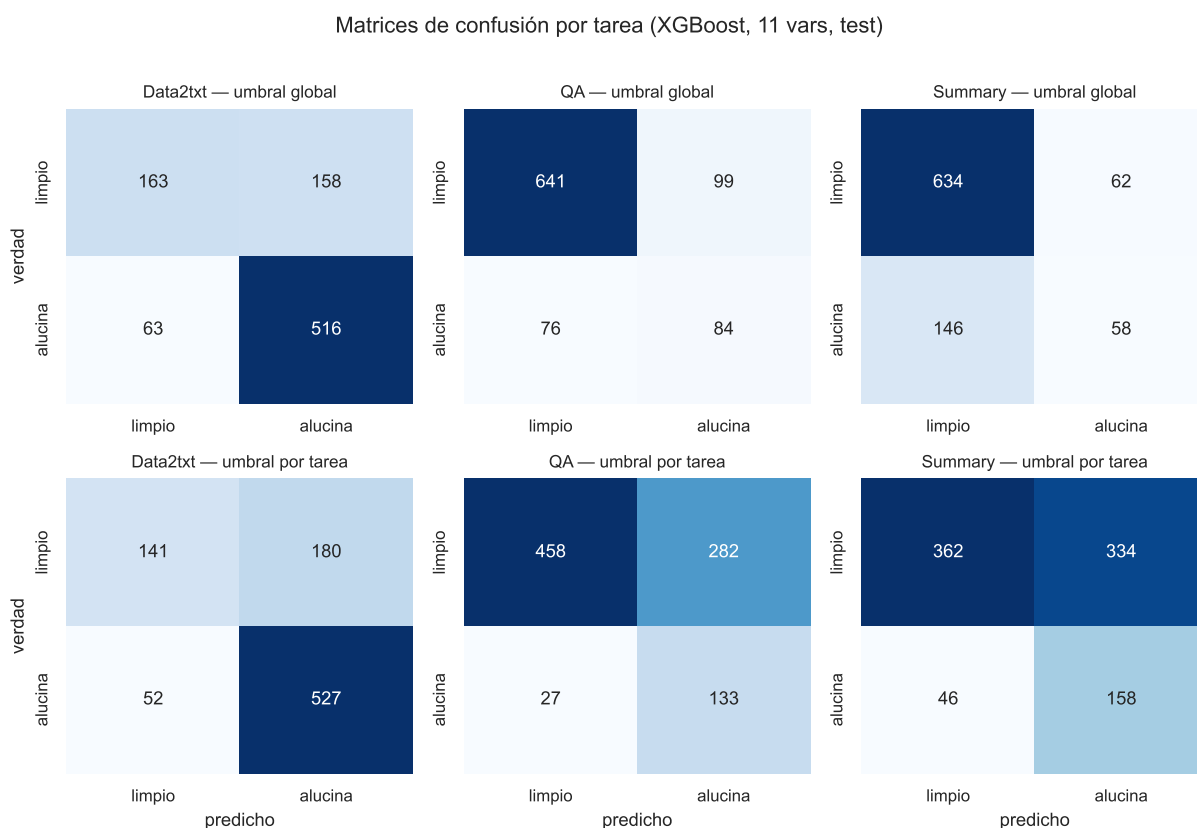


Figura 1.21: Matrices de confusión por tarea (XGBoost, test). Fila superior: umbral global; fila inferior: umbral por tarea. La detección de alucinaciones (VP) crece en QA y Summary.

Método	Tipo	Parámetros	Hardware	Coste/eval
GPT-4-turbo (juez)	LLM-juez	$\sim 10^{12}$ (est.)	API	$\sim 10^{-2}$ €
Luna	encoder ligero	$\sim 0,4$ B	GPU	$\sim 10^{-4}$ €
LettuceDetect-large	encoder (ModernBERT)	0,396 B	GPU	$\sim 10^{-4}$ €
Fine-tuned Llama-2-13B	LLM fine-tuned	13 B	GPU	$\sim 10^{-3}$ €
Lynx-8B / 70B	LLM fine-tuned	8–70 B	GPU(s)	10^{-3} – 10^{-2} €
RAG-HAT	LLM fine-tuned	(grande)	GPU	$\sim 10^{-3}$ €
Este trabajo	ML clásico	$\sim 10^2$	CPU	≈ 0

Tabla 1.13: Los métodos de referencia sobre RAGTruth, por tipo, tamaño, hardware y coste. El detector clásico opera en un régimen de coste distinto: CPU, sin parámetros aprendidos por millones, inferencia gratuita y determinista.

1.8.2. Tabla comparativa

El estado del arte se mide en F1 a nivel de respuesta y en accuracy, no en AUC (la literatura de RAGTruth no reporta AUC). El Cuadro 1.14 sitúa nuestro detector por tarea y la Figura 1.22 lo grafica.

Nota de comparabilidad. Todas las cifras son sobre RAGTruth, split oficial y F1 a nivel de respuesta, pero los métodos de la literatura son *neuronales o basados en LLM*, mientras el nuestro es clásico. La tabla de accuracy (Lynx) se reporta sobre el subconjunto de RAGTruth incluido en HaluBench, no sobre el split idéntico. Se comparan, por tanto, cifras sobre el mismo problema y benchmark, no el mismo tipo de modelo.

Método (F1 nivel respuesta, %)	QA	Data2txt	Summary	Global
GPT-4-turbo (juez)	45,6	78,3	47,6	63,4
Luna (encoder ligero)	51,3	75,9	52,5	65,4
LettuceDetect-base (150M)	65,5	87,9	50,5	76,1
Fine-tuned Llama-2-13B	68,2	88,1	59,1	78,7
LettuceDetect-large (396M)	70,2	88,5	59,7	79,2
RAG-HAT (mejor)	74,8	91,6	67,6	83,9
Este trabajo (clásico)	46,3	82,0	45,4	68,5

Tabla 1.14: F1 por tarea sobre RAGTruth, recopilado por Kovács et al. [13] con RAG-HAT [21] como cota alta. Nuestro F1 por tarea usa umbral por tarea; el global (68,5) es el F1 *pooled* con umbral de Youden (el F1 macro por tarea es 55,7).

1.8.3. Conclusión: precio, calidad y complejidad

En accuracy, la referencia es Lynx [18]: GPT-3.5-turbo 50,7 %, Claude-3-Sonnet 79,1 %, Lynx-8B 80,0 %, Lynx-70B 80,2 % y GPT-4o 84,3 %; nuestra accuracy es 77,6 %. La lectura honesta cruza las tres dimensiones del Cuadro 1.13. En **calidad**, superamos en F1 global al juez GPT-4-turbo (63,4) y a Luna (65,4), y quedamos unos diez puntos por debajo de los modelos neuronales ajustados (79–84). En **complejidad y precio**, la distancia es inversa y enorme: frente a miles de millones de parámetros, GPU y un coste por evaluación

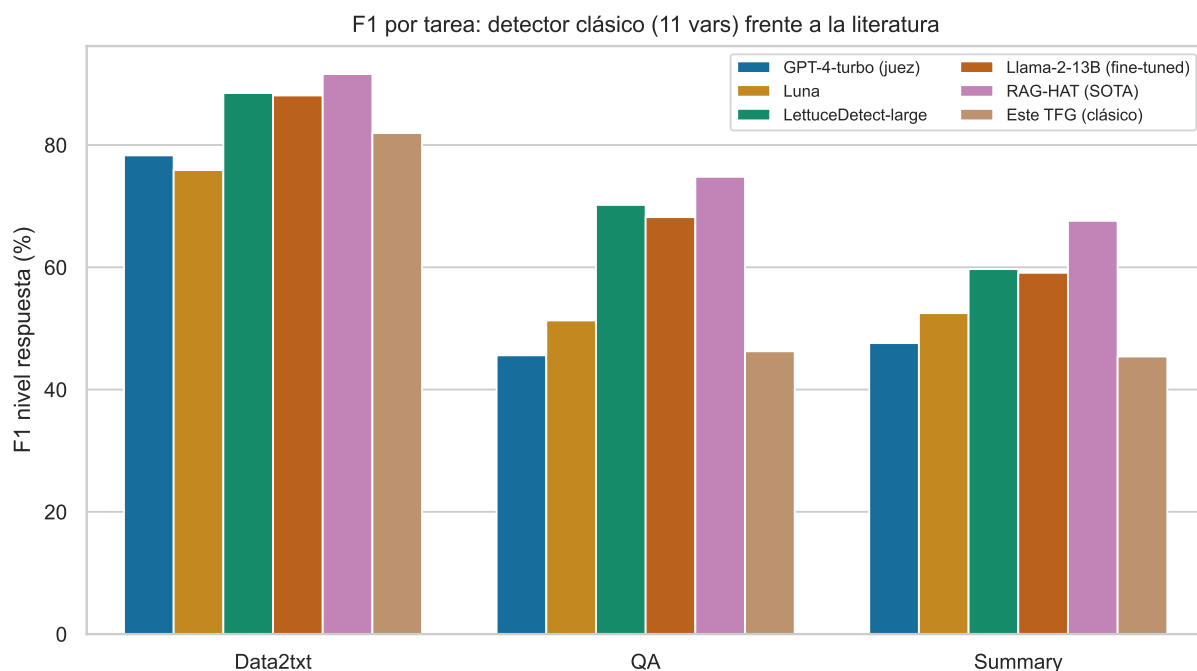


Figura 1.22: F1 por tarea: nuestro detector clásico frente a la literatura. La brecha se concentra en Summary, la tarea que nadie resuelve con solvencia.

no nulo, nuestro detector usa un centenar de parámetros, corre en CPU, es determinista, interpretable y gratis. Y el patrón por tarea es el mismo para todos: Data2txt fácil (88–92%), QA y Summary mucho más bajos. Chocamos contra el mismo muro que modelos mil veces mayores, solo un poco antes. La aportación no es ganar en calidad absoluta, sino mostrar cuánta se alcanza a coste esencialmente nulo, y *acotar con precisión el techo* de lo léxico.

1.9. Exploración: vías descartadas

El resultado anterior es el destilado de una exploración amplia. Se recogen aquí, de forma condensada, las vías que no funcionaron, porque forman parte de la caracterización del problema y no del hilo principal.

Características que no aportaron. Se probaron y descartaron, todas medidas con el protocolo honesto: LSA (TF-IDF con descomposición en valores singulares truncada [7]), que aportó +0,001 de AUC; la novedad de n -gramas por frase, +0,001; y características estructurales de spaCy (arcos de dependencia, sujeto-verbo-objeto, enlace entidad-número), que dejaron Summary en $0,701 \rightarrow 0,700$. Este último resultado es informativo: ni siquiera features estructurales clásicas cierran Summary, lo que confirma que el residuo es semántico y de conocimiento del mundo, fuera del alcance de la superficie.

Tratamiento del desbalance que no movió la frontera. Reequilibrar las clases no crea señal: el sobremuestreo, el submuestreo, SMOTE [2] y el aprendizaje sensible al coste dejan la frontera (AUC-PR por tarea) plana o peor (Figura 1.23a). La corrección de prior de Saerens [19] colapsa: su estimación por esperanza-maximización converge a prevalencias

degeneradas (0 o 1) (Figura 1.23b). La única palanca honesta contra el falso negativo es el umbral por tarea de la sección 1.5.4.

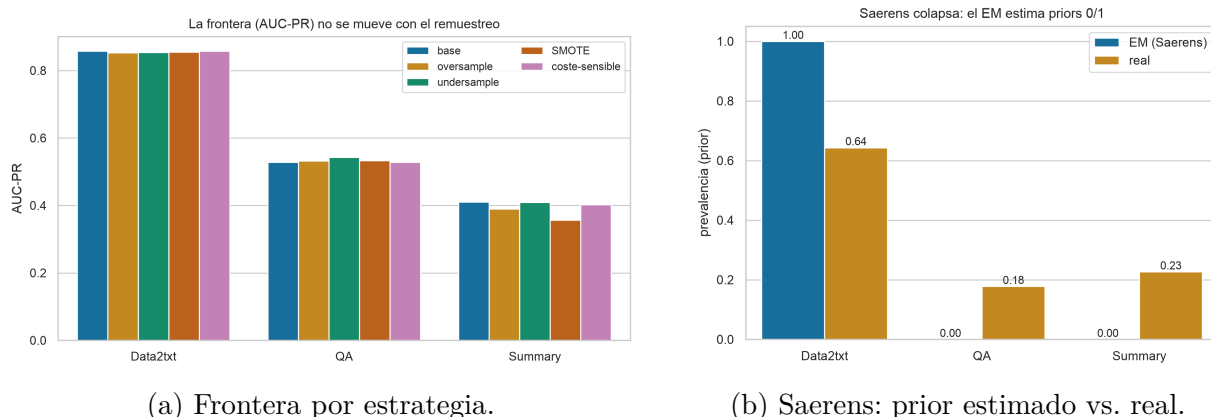


Figura 1.23: Reequilibrar clases no mueve la frontera; la corrección de prior de Saerens colapsa. Resultados negativos, documentados por honestidad.

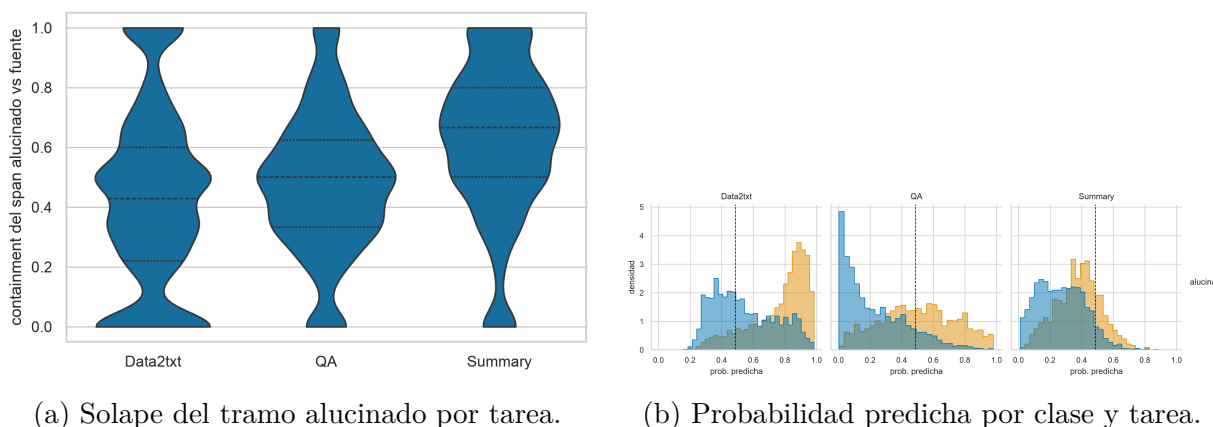


Figura 1.24: El cuello de botella de Summary es el camuflaje léxico, no el tipo de alucinación.

1.10. Conclusiones

Hipótesis. Se cumple. Los cinco clasificadores clásicos superan el AUC-ROC de 0,80 sobre el test oficial con evaluación honesta, y el mejor, XGBoost, alcanza 0,837 con solo once características léxicas seleccionadas por RFE.

Falsos negativos y desbalance. El déficit de detección no era de características ni de balance de clases, sino del punto de decisión bajo un desbalance heterogéneo. Un umbral por tarea recupera casi la mitad de los falsos negativos (de 285 a 125) y hace competitivo el F1 de Summary. La corrección de prior, el remuestreo y el aprendizaje sensible al coste se descartaron con evidencia.

Posicionamiento. El detector rinde por encima del juez GPT-4-turbo y de Luna, y a unos diez puntos de los detectores neuronales ajustados, a coste cero, en CPU, de forma interpretable y determinista. El techo de Summary queda *caracterizado, no supuesto*: es la frontera entre la minería léxica de superficie y la semántica profunda, y el estado del arte, con modelos mucho mayores, tropieza con el mismo muro.

Bibliografía

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1), 2001.
- [2] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 2002.
- [3] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [4] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3), 1995.
- [5] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 1967.
- [6] David R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2), 1958.
- [7] Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 1990.
- [8] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 2006.
- [9] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 2001.
- [10] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 2003.
- [11] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46(1–3), 2002.
- [12] Ziwei Ji, Nayeon Lee, Rita Frieske, et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.
- [13] Ádám Kovács and Gábor Recski. LettuceDetect: A hallucination detection framework for RAG applications. *arXiv preprint arXiv:2502.17125*, 2025.

- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [15] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [16] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 1999.
- [17] Por completar. Pendiente: referencia sobre la falta de fiabilidad del llm como juez (sustituir), 2026. Placeholder: indicar el paper concreto sobre fallos del LLM-juez.
- [18] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, et al. Lynx: An open source hallucination evaluation model. *arXiv preprint arXiv:2407.08488*, 2024.
- [19] Marco Saerens, Patrice Latinne, and Christine Decaestecker. Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1), 2002.
- [20] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 1988.
- [21] Juntong Song, Xingguang Wang, Juno Zhu, et al. RAG-HAT: A hallucination-aware tuning pipeline for LLM in retrieval-augmented generation. In *Proceedings of EMNLP 2024 (Industry Track)*, 2024.
- [22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.